SUMMARY: RAG WITH LANGCHAIN

By the end of this session, students will be able to:

1. Summarize the universal RAG pipeline and map each manual step to the equivalent LangChain component.
2. Ingest HTML documents, chunk them, embed with HuggingFace, and index into Pinecone via LangChain.
3. Configure dense, sparse, and hybrid retrievers with k, filters, and namespaces.
4. Apply query-transformation patterns (multi-query, self-query, decomposition, normalization) to boost recall.
5. Re-rank retrieved candidates with a cross-encoder and select a compact, diverse evidence set.
6. Clean, compress, deduplicate, and context-budget passages for efficient prompting.
7. Construct robust prompts with ChatPromptTemplate and enforce citation style and output structure.
8. Add and manage conversation history separately from documents using RunnableWithMessageHistory.
9. Build a Memory-RAG that merges long-term docs with user memory and cites sources reliably.
10. Implement a minimal end-to-end RAG loop (retrieve → prepare → prompt → generate → verify) in ≤5 notebook cells.

The Common Structure of RAG Pipelines

No matter whether we looked at Memory RAGs, Branched RAGs, Adaptive RAGs, or Corrective RAGs, the skeleton beneath them all is surprisingly similar. Each variation only adds a layer on top. Let's carefully look at what is always common, and what we had to do manually.

1. The Universal Skeleton of a RAG

1.1 Ingest and Store

Every RAG starts with knowledge ingestion. We load our documents (in our case, the coffee HTML pages), extract clean text, optionally chunk into smaller parts, create embeddings, and push them into a vector database. Alongside embeddings, we attach metadata such as titles, headings, keywords, and timestamps. This creates a searchable knowledge store.

1.2 Query Transformation

When a user asks a question, we don't throw it raw at the system. Instead, we embed it into a dense vector representation. Sometimes we also keep a sparse representation like BM25 for exact word matches. The idea is: represent the query in the same language as the knowledge store.

1.3 Retrieval of Candidates

The vector database is then queried. Dense retrieval finds semantically close matches. Sparse retrieval ensures keyword precision. Hybrid retrieval blends both. Optionally, we apply metadata filters (for example, limiting results only to coffee types with "ashwagandha" in their heading).

1.4 Context Preparation

From all retrieved documents, we select the most relevant few. We normalize scores, fuse dense and sparse rankings, maybe re-rank with a neural model, and finally pick a handful of passages. These are then cleaned and shortened, so they fit within the LLM's context window.

1.5 Prompt Construction

The chosen passages, along with the user's past conversation and relevant memory, are inserted into a prompt template. The template clearly separates user input, retrieved context, and past LLM outputs. This ensures the model knows what came from where.

1.6 Generation and Answering

The LLM uses both its pretrained knowledge and the injected context to generate an answer. Because it is grounded in actual documents, hallucinations are reduced. The response is then returned to the user.

1.7 Optional Correction

Some advanced pipelines, like Corrective or Self-RAG, add one more step: validate if the answer was grounded. If not, re-retrieve or regenerate. This ensures higher trustworthiness.

2. What We Implemented Manually

2.1 Parsing Documents

We used BeautifulSoup to parse HTML pages, stripping unnecessary tags and keeping only headings and text. We ourselves decided what parts matter.

2.2 Metadata Handling

We added metadata such as `title`, `headings`, `heading_keywords`, and `timestamps` during ingestion. This gave us the power to later filter queries.

2.3 Embedding Generation

We selected a SentenceTransformer model (`MiniLM`), encoded each page manually, and made sure dimensions matched our index.

2.4 Sparse Index Creation

When we wanted hybrid search, we built a BM25 index separately. This meant we maintained two retrieval pipelines ourselves.

2.5 Retrieval Fusion

We manually normalized dense and sparse scores, and fused them with a weighted sum (`α * dense + (1-α) * sparse`). This required experimentation and tuning.

2.6 Prompt Building

We manually wrote prompt templates in `.yaml` or `.md`, clearly separating user inputs, memory, and retrieved text. Every LLM call was manually formatted.

2.7 Memory Management

For memory-based RAGs, we manually carried forward prior interactions. We also discussed the problem of infinite growth and how summarization or recency selection solves it.

2.8 Quality Control

Whenever results were poor, we manually debugged — maybe widened the filter, adjusted weights, or restructured prompts.

3. Why This Matters

Doing all of this manually showed us the **inner plumbing** of RAG systems.

- We understood why metadata is crucial.
- We saw why sparse + dense fusion improves recall.
- We realized that prompts must be cleanly structured, else the model confuses user input with retrieved text.
- And we experienced the pain points: score normalization, memory growth, prompt explosion.

This sets the stage for introducing frameworks like **LangChain** later — because now we know exactly what moving parts they abstract away.

So, while all the RAGs we've built look different — Memory RAG, Adaptive RAG, Corrective RAG — they all share the **same skeleton**. The differences come in *retrieval strategy* and *post-generation handling*.

Introduction to LangChain

LangChain is not "just another library." It's a **framework** for building end-to-end LLM-powered applications. Think of it as the connective tissue that ties together models, data sources, memory, retrieval systems, tools, and workflows — so you don't have to rebuild them every time.

1. Why LangChain Exists

When you try to build an AI system around LLMs manually, you run into repeating problems:

- How do I keep and manage **memory** across turns?
- How do I connect my LLM to **external data** (databases, APIs, vector stores)?
- How do I structure **retrieval pipelines** without rewriting boilerplate?
- How do I orchestrate **multi-step reasoning** (like branched or adaptive workflows)?
- How do I test, trace, and deploy these pipelines?

LangChain emerged as an abstraction layer to solve these problems. Instead of engineers writing custom glue code every time, LangChain provides **ready-made components** and **composable building blocks**.

2. Core Building Blocks of LangChain

1. **LLM Wrappers**
   A unified interface to call different models (OpenAI, Gemini, Anthropic, HuggingFace). You swap models without rewriting your logic.
2. **Prompt Templates**
   Instead of building raw strings, you define templates with placeholders (`{context}`, `{question}`) and LangChain fills them in. They support chaining, variables, and structured formatting.
3. **Chains**
   Chains are pipelines of steps. For example:
   - Query → Retriever → Prompt → LLM → Output.

- Or multi-step: Input → Tool → Retriever → LLM → Validator → Answer.
  Chains capture the **logic flow** of your application.

4. **Retrievers**
   Specialized objects that fetch relevant documents given a query. These can be dense (vector DBs like Pinecone, FAISS), sparse (BM25, ElasticSearch), hybrid (EnsembleRetriever), or even self-querying (where the LLM decides filters automatically).
5. **Vector Stores**
   Integrations with Pinecone, Weaviate, FAISS, Milvus, Chroma, and others. LangChain makes storing and retrieving embeddings uniform.
6. **Memory**
   Out of the box, LangChain offers memory types:

   - ConversationBufferMemory (stores everything)
   - ConversationSummaryMemory (summarizes older history)
   - ConversationKGMemory (stores facts in a knowledge-graph form)
     These avoid us having to manually manage user/assistant history.
7. **Agents**
   The most powerful abstraction. An agent uses the LLM not just as a responder, but as a *decision-maker*. The LLM decides what tool to call (a retriever, an API, a calculator), gets results, and decides the next step. This lets you build **adaptive, tool-using AI agents** without hardcoding logic.
8. **Toolkits and Integrations**
   Ready-made tools: Google search, SQL DB connectors, APIs, file loaders. These allow you to build domain-specific agents without writing every adapter yourself.
9. **Evaluation and Tracing**
   LangChain integrates with LangSmith for experiment tracking, tracing, dataset-based evaluation, latency analysis, and cost monitoring.

If manual RAG is like assembling furniture plank by plank with screws, LangChain is like IKEA: you still build it, but with standardized parts, instructions, and the option to add extensions.

It reduces boilerplate, increases speed, and makes experiments easier — while still leaving room for custom control when needed.

In short, LangChain abstracts all the **manual steps we coded** (embedding, retrieval, prompt formatting, memory, fusion) into composable components. But it also **goes beyond**, enabling agents, multi-step reasoning, and tool integration.

LangChain Architecture for RAGs

1. The Core Architectural Principles

LangChain is not about giving you *new tricks an LLM doesn't already know*. Instead, it's about **organizing the chaos** when you want to connect an LLM to the real world.Think of a **manual RAG pipeline** you've already built:

- Step 1: take a user question
- Step 2: search in a vector DB
- Step 3: stuff retrieved docs into a prompt
- Step 4: call the LLM

That's fine for a quick prototype. But as soon as you want to add memory, swap retrieval methods, run self-checks, or connect extra tools, the spaghetti code begins.

LangChain's architecture prevents this by splitting the world into a few **clear layers of abstraction**:

- **Model I/O Layer** – a universal "plug" to call any LLM in the same way.
- **Data Connection Layer** – standard interfaces for retrievers, vector stores, and data loaders.
- **Memory Layer** – keeping track of conversations or previous retrievals
- **Chains Layer** – reusable pipelines (like `RetrievalQAChain`) that combine retrievers, prompts, and LLMs.
- **Agents Layer** – letting the model decide which tool/retriever/pipeline to use dynamically.

Each of these isn't just theory — they're meant to swap in and out easily. Want FAISS today and Pinecone tomorrow? Just switch the retriever, nothing else changes.

2. How These Modules Work in RAGs

Let's now walk through them, but specifically with **RAG in mind**.

- **Model I/O:** Your brain (LLM) can be GPT, Claude, Gemini, or a local model. LangChain makes them all look the same so you can experiment quickly.
- **Data Connections:** This is where LangChain shines for RAG. You can:
    - Use a single retriever (say Pinecone).
    - Use hybrid retrievers (BM25 + embeddings).
    - Even combine retrievers into a pipeline.
- **Memory:** Manual RAGs often miss this. In LangChain, memory can store:
    - What was asked before, so you don't re-fetch the same info.
    - What was answered, so context flows naturally across turns
- **Chains:** A chain is simply a **workflow in a box**. For RAG, the most common is `RetrievalQAChain`:

```
User question → Retriever → PromptTemplate → LLM → Answer
```

But you can nest chains: e.g., retrieval → summarization → answer.

- **Agents:** This is the advanced layer. Instead of a rigid RAG, you give the model a "toolbox":
    - Retrieval tool
    - SQL tool
    - Calculator
    - Web search

3. Why This Architecture Matters

LangChain's design is really about **scaling beyond the prototype**. Here's why it matters:

- **Modularity:** Swap components (retrievers, LLMs, prompts) without rewriting code.
- **Experimentation:** Test retrieval strategies, evaluate them, and monitor with LangSmith.
- **Statefulness:** Memory makes RAGs multi-turn and more human-like.
- **Adaptivity:** Agents allow RAGs to decide *how much to retrieve* or *which tool to use*.
- **Orchestration:** Built-in caching, retries, logging, and evaluation let you think less about plumbing and more about product.

So, the big shift is:

- Manual RAG = *a clever demo*.
- LangChain RAG = *a structured, extensible system you can actually deploy and evolve*.

Setting up a LangChain RAG Pipeline

So far we have seen how to build RAG pipelines manually, wiring embeddings, vector databases, retrieval, prompt construction, and memory all by ourselves. This gave us full control but also meant lots of repetitive boilerplate. Now we shift gears to see how **LangChain simplifies these steps**. The idea is to show how the same workflow can be assembled quickly and more cleanly, while still allowing us to customize every part.

We will walk through all nine structural steps of RAG, but this time inside the LangChain framework. Our dataset will be the same **coffee HTML pages**, so students can clearly see the parallel.

1. Ingest and Store Knowledge

1.1 Loading Documents

LangChain has many **document loaders** that abstract away the parsing of raw files. Instead of writing BeautifulSoup code ourselves, we can plug in loaders for HTML, PDFs, Notion, Slack, or Google Docs. Here we'll still demonstrate with our HTML files so the pipeline remains consistent.

```
# pip install -U langchain-community unstructured bs4

from langchain_community.document_loaders import DirectoryLoader,
UnstructuredHTMLLoader

# Load all HTML files from a folder into LangChain Documents
loader = DirectoryLoader(
    "coffee_pages",
    glob="**/*.html",
    loader_cls=UnstructuredHTMLLoader
)
docs = loader.load()
print("Loaded:", len(docs))
```

1.2 Splitting and Embeddings

Manually we had to chunk text and embed it. In LangChain, chunking is handled by **TextSplitters** and embedding by **Embedding wrappers**.

Document Splitting in LangChain

- LangChain provides a suite of TextSplitters to break documents into chunks, using strategies like length-based (token or character count), structure-based (paragraphs, sentences), and semantic-based (meaningful context).
- Advanced splitters like RecursiveCharacterTextSplitter try to keep natural groupings—paragraphs or sentences—intact, only splitting smaller as needed, resulting in chunks that preserve context.
- Splitters can also introduce chunk overlap: a bit of shared content between consecutive chunks, reducing boundary-context loss and helping downstream models understand content better.

Embedding Workflow in LangChain

- After splitting, each chunk is passed through an embedding model (e.g., OpenAI, HuggingFace), transforming text into high-dimensional vectors suitable for semantic search and retrieval.
- LangChain abstracts embedding interfaces, making it easy to swap providers. Methods like embed_documents() and embed_query() generate document and query embeddings, respectively.
- The resulting vectors are stored in a vector store (such as FAISS, Chroma), which supports fast similarity search, enabling retrieval-augmented generation and question-answering tasks over large corpora

```
# pip install -U sentence-transformers

from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_google_genai import GoogleGenerativeAIEmbeddings
# Split into chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
chunks = splitter.split_documents(docs)
print("Chunks:", len(chunks))
```

```
# Define embeddings
embedding = HuggingFaceEmbeddings(
    model_name="sentence-transformers/paraphrase-MiniLM-L6-v2"
)
```

1.3 Storing into Pinecone

LangChain provides a **PineconeVectorStore** integration that handles upserts, metadata, and namespace management.

How LangChain Stores in Pinecone

- **Initialization:** You first connect to an existing Pinecone index or create one. LangChain's PineconeVectorStore requires this index along with an embedding model (e.g., OpenAI embeddings) to be initialized.
- **Storing Documents:** LangChain allows adding documents or raw text to the Pinecone index by passing them to methods like *from_documents* or *from_texts*. These methods internally split documents if necessary, compute embeddings, and then insert the vectorized representations with optional metadata into Pinecone.
- **Adding Data Incrementally:** Additional documents or texts can be added post-initialization using methods such as *add_documents* or *add_texts*. These handle embedding creation and storing vectors in Pinecone automatically.
- **Querying:** LangChain wraps Pinecone's similarity search with methods like *similarity_search* that help retrieve the most semantically relevant documents or vectors based on a text query's embedding.
- **Metadata and IDs:** Documents stored in Pinecone can have associated metadata and unique IDs to manage and delete entries selectively, supporting flexible data lifecycle management.

```
from pathlib import Path
import os
from pinecone import Pinecone, ServerlessSpec

os.environ["PINECONE_API_KEY"] =
Path("api_key.txt").read_text().strip()
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])

INDEX_NAME = "coffeeindex"
REGION     = "us-east-1"
```

```
EMBED_DIM  = 384

# (Re)create
if INDEX_NAME in pc.list_indexes().names():
    pc.delete_index(INDEX_NAME)
pc.create_index(
    name=INDEX_NAME,
    dimension=EMBED_DIM,
    metric="cosine",
    spec=ServerlessSpec(cloud="aws", region=REGION),
)
idx = pc.Index(INDEX_NAME)
idx
```

```
from langchain_community.embeddings import HuggingFaceEmbeddings

# MiniLM-L6-v2 → 384-dim
embedding = HuggingFaceEmbeddings(
    model_name="sentence-transformers/paraphrase-MiniLM-L6-v2"
)

# Sanity: check embedding size
vec = embedding.embed_documents(["hello world"])[0]
print("HF embed dim:", len(vec))  # expect 384
```

```
from langchain_pinecone import PineconeVectorStore

vectorstore_from_docs = PineconeVectorStore.from_documents(
    chunks,      # your split LangChain Documents
    embedding=embedding,    # HF embedder (384)
    index_name=INDEX_NAME,
    text_key="text",
)
```

At this point, we have a **structured vector index** with embeddings, metadata, and chunked coffee texts — all without hand-wiring BeautifulSoup parsing, embedding loops, or Pinecone upserts.

2. Query Transformation

The query is the steering wheel of your RAG. Before we search the vector DB, we can reshape the user's text to make retrieval sharper, cheaper, and more robust. In manual RAG you wrote this glue yourself (regexes, Pinecone filters, BM25 calls, score fusion). In LangChain you compose small, reusable "transformers" and drop them in front of a retriever.

2.1 What and why (manual vs LangChain)

Idea
We alter the query—or what we embed on its behalf—so retrieval brings back the *right* passages. Typical moves: add metadata filters, generate paraphrases, build a hypothetical answer to embed (HyDE), infer filters from the text, decompose multi-facet questions, normalize units/spelling, or translate.

Manual vs LangChain

Manual: you embedded the raw query, hand-built Pinecone `filter={...}`, ran BM25 yourself, normalized and fused scores, and wrote separate code paths per case.
LangChain: you wrap your base retriever with transformation utilities (e.g., `MultiQueryRetriever`, `SelfQueryRetriever`) or pre-process the text with small chains; LangChain takes care of calling the retriever and merging results.

2.2 Patterns and recipes (with side-by-side comparisons)

Assumptions: you already created `vectorstore = Pinecone(..., namespace="docs")`, and we'll use `retriever = vectorstore.as_retriever(search_kwargs={"k": 8})`. We'll also use a Gemini wrapper: `llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", google_api_key=GEMINI_API_KEY)`.

a. Pass-through baseline

- **Embedding Initialization:** It creates an embedding model instance using HuggingFace's "sentence-transformers/paraphrase-MiniLM-L6-v2" which converts text into dense vector representations for semantic similarity.
- **Vector Store Setup:** It initializes a Pinecone vector store named "coffeeindex" within namespace "docs", connecting it to the embedding model and specifying "text" as the metadata key for document content storage.
- **Retrieval and Query:** It creates a retriever from the vector store configured for similarity search with top 8 results, then queries this retriever with "What is turmeric coffee?" to fetch the 8 most semantically relevant documents, printing their count.

```
# pip install -U pinecone langchain-pinecone langchain-community
sentence-transformers

from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_pinecone import PineconeVectorStore

embedding =
HuggingFaceEmbeddings(model_name="sentence-transformers/paraphrase-Mi
niLM-L6-v2")

vectorstore = PineconeVectorStore(
    index_name="coffeeindex",
    embedding=embedding,
    namespace="docs",
    text_key="text",   # be explicit
)

retriever = vectorstore.as_retriever(search_type="similarity",
search_kwargs={"k": 8})
docs = retriever.get_relevant_documents("What is turmeric coffee?")
print(len(docs))
```

b. Keyword → metadata filters (cheap, high signal)

How Metadata Filters Work in Pinecone

- Each vector in Pinecone can have associated metadata as key-value pairs (e.g., category, date, user ID), with values being strings, numbers, or booleans.
- When performing a similarity search, users can specify filter expressions that restrict the search to only those vectors whose metadata matches the provided conditions. This enables focused and contextually relevant retrieval.

Benefits of Pinecone Metadata Filters

- Enables precise narrowing down of results within large datasets by combining similarity search with metadata constraints.
- Supports complex logical filters using operators such as equality, inequality, greater/less than, exists, and logical AND/OR to build fine-tuned queries.
- Improves relevance for use cases like filtering by document type, language, category, user context, or recency — critical for retrieval augmented generation (RAG) systems and AI applications.

Efficiency and Accuracy

- Pinecone uses a single-stage filtering mechanism that merges vector similarity and metadata filtering into one efficient process, avoiding the slower two-step filter-then-search approach.
- This design maintains high accuracy and speed even at scale, allowing metadata filtering on datasets with millions to billions of vectors without significant performance loss

```
# Search with automatic Keyword detection
import os
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.chains.query_constructor.base import AttributeInfo
from langchain.retrievers import SelfQueryRetriever

# Ensure Gemini key is in env
if "GEMINI_API_KEY" not in os.environ:
    with open("gemini_key.txt") as f:
        os.environ["GEMINI_API_KEY"] = f.read().strip()

# 1. Define metadata schema fields
metadata_field_info = [
    AttributeInfo(
        name="heading_keywords",
        description="Main topics or section headings in the document
(e.g., Turmeric, Ashwagandha, Saffron, Benefits, Side Effects)",
```

```
        type="string or list[string]",
    ),
    AttributeInfo(
        name="department",
        description="Business department such as HR, Finance,
Marketing, or Support",
        type="string",
    ),
]

# 2. Gemini LLM for query rewriting + filter extraction

llm_filter = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],  # 👈 force API key
auth
    temperature=0,
)


# 3. Build a SelfQueryRetriever
retriever = SelfQueryRetriever.from_llm(
    llm=llm_filter,
    vectorstore=vectorstore,   # your Pinecone LangChain vectorstore
    document_contents="Knowledge base about coffee, herbs, and health
benefits.",
    metadata_field_info=metadata_field_info,
    enable_limit=True,         # allow Gemini to set 'k' dynamically
)

# 4. Example query -- Gemini auto-adds filter (e.g.
heading_keywords="turmeric")
query = "Ashwagandha?"
docs = retriever.get_relevant_documents(query)

print(f"\nSelfQueryRetriever returned {len(docs)} docs")
for d in docs[:5]:
    print("-", d.metadata.get("headings"), "| chunk:",
d.metadata.get("chunk_id"))
```

c. Multi-query expansion (paraphrases for recall)

- Multiquery expansion in LangChain leverages large language models to enhance the original search query by generating multiple related or refined queries. This process broadens the scope of retrieval and helps capture more relevant and diverse documents that may be semantically related but do not match the original query exactly.

- By expanding a single input query into several context-aware variants, the system can perform multiple searches and then merge and deduplicate results. This increases recall—finding more potentially relevant documents—while maintaining precision by filtering out redundancies. The approach helps AI systems understand user intent better, especially when the original query is ambiguous or narrowly phrased.

- Ultimately, multiquery expansion strengthens semantic search by combining the power of LLMs for query reformulation with vector search retrieval, leading to richer and more comprehensive information access. This technique aligns well with retrieval-augmented generation (RAG) workflows where capturing a wide but relevant knowledge base is crucial.

Manual: call LLM for rewrites, loop over Pinecone queries, merge and dedupe IDs.

LangChain:

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.retrievers import MultiQueryRetriever

# LLM for query expansion
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0
)
```

```
# Wrap your Pinecone vectorstore
base_retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

# MultiQueryRetriever: uses LLM to expand queries, merges + dedupes
results
multi_retriever = MultiQueryRetriever.from_llm(
    retriever=base_retriever,
    llm=llm,
    include_original=True,   # keep original query too
)

query = "What are the health benefits of turmeric coffee?"
docs = multi_retriever.get_relevant_documents(query)

print(f"Got {len(docs)} unique docs with multi-query expansion")
for d in docs[:5]:
    print("-", d.metadata.get("headings"), "| file:",
d.metadata.get("file_name"))
```

LangChain handles generating variants and merging results for you.

d. Self-Query Retriever (infer filters automatically)

The SelfQueryRetriever in LangChain is an advanced retrieval mechanism that uses a large language model (LLM) to intelligently convert a natural language user query into a structured query that the underlying vector store can understand, enabling highly relevant and context-aware document retrieval.

How SelfQueryRetriever Works

- It takes in metadata field information (like document attributes such as keywords or timestamps) along with a description of the document contents. This metadata info helps the LLM construct precise filters based on user queries.
- The LLM parses the natural language query and generates a structured query that includes metadata filters and content search terms, effectively translating vague user questions into exact retrieval conditions.
- The vector store then executes this structured query—combining semantic similarity with metadata filtering—to return documents that are both relevant in content and contextually filtered by attributes like recentness or topic keywords.

Key Benefits

- Natural language query understanding: Users can ask complex questions involving metadata constraints (e.g., "Show recent saffron coffee entries") without needing to write formal queries.
- Dynamic filtering: Enables real-time generation of filters and limits without manual query crafting.
- Better precision: Results consider both semantic text similarity and structured metadata filters together for improved accuracy.

In essence, LangChain's SelfQueryRetriever bridges the gap between rich natural language queries and the structured, attribute-driven search capabilities of vector databases, making retrieval more intuitive, precise, and powerful.

Manual: you wrote regexes or custom parsers to populate `filter={...}`.
LangChain:

```
# pip install lark
from langchain.retrievers.self_query.base import SelfQueryRetriever
from langchain.chains.query_constructor.schema import AttributeInfo
from langchain.retrievers import MultiQueryRetriever

from langchain_google_genai import ChatGoogleGenerativeAI

# Gemini LLM via LangChain wrapper
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0,
)

metadata_field_info = [
    AttributeInfo(
        name="heading_keywords",
        description="Lowercased keywords present in headings (e.g.
turmeric, saffron, ashwagandha)",
        type="list[string]",
    ),
    AttributeInfo(
        name="ts",
        description="Unix ingestion timestamp (seconds since epoch)",
        type="integer",
    ),
```

```
]

selfq = SelfQueryRetriever.from_llm(
    llm=llm,   # 👈 wrapper, not genai.GenerativeModel
    vectorstore=vectorstore,
    document_contents="Indian coffee recipes and variants",
    metadata_field_info=metadata_field_info,
    enable_limit=True,
)

docs = selfq.get_relevant_documents("Show recent saffron coffee
entries")
print(f"Got {len(docs)} docs")
for d in docs[:5]:
    print("-", d.metadata.get("headings"), "| ts:",
d.metadata.get("ts"))
```

e. Sub-question decomposition (facet coverage)

- Subquestion decomposition is a technique where a complex or multi-faceted natural language query is broken down into several smaller, more focused sub-questions that collectively answer the original query. This method helps improve retrieval coverage and answer accuracy by enabling separate, targeted searches for each aspect of the query.

- By decomposing a broad question into 2–4 concise sub-questions, the system can independently retrieve relevant documents for each, then merge and deduplicate results. This approach leverages the reasoning ability of large language models to understand underlying parts or components of a user's complex request, breaking it down into simpler semantic units. Each sub-question is easier to handle for vector search or question-answering components, resulting in more comprehensive and precise responses.

- Subquestion decomposition also limits noise by isolating specific information needs, improving precision over querying only with the original complex question. It's commonly used in retrieval-augmented generation (RAG) to enhance knowledge retrieval and reasoning workflows by distributing the semantic load across multiple focused retrievals rather than a single monolithic one.

- In summary, subquestion decomposition helps AI systems transform a complex user query into manageable chunks, facilitating better coverage, relevance, and final answer quality through distributed query reasoning.

Manual: call LLM to split, loop over Pinecone, then union and dedupe IDs.
LangChain:

```
from langchain.prompts import PromptTemplate
from langchain.retrievers.multi_query import MultiQueryRetriever

decomp_prompt = PromptTemplate.from_template(
    "Decompose into 2-4 concise sub-questions that together answer:
{question}\nReturn one per line."
)

multi = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
    llm=llm,
    prompt=decomp_prompt,     # your custom decomposition style
    include_original=True,    # also query with the original question
)

q = "Compare turmeric coffee vs saffron coffee for health effects and
taste"
docs = multi.get_relevant_documents(q)   # <- already merged &
deduped
for d in docs[:5]:
    print("-", d.metadata.get("headings"), "| ts:",
d.metadata.get("ts"))
```

f. Multilingual handling (detect/translate → retrieve

Translate non-English queries into the corpus language before retrieval.

Manual: detect language → call translate API → re-query Pinecone.
LangChain:

```
from langchain.chains import LLMChain
```

```
from langchain.prompts import PromptTemplate

translate_prompt = PromptTemplate.from_template("Translate to
English, keep meaning: {text}")
translate = LLMChain(llm=llm, prompt=translate_prompt)

raw_q = "क्या केसर कॉफी मीठी होती है?"  # Hindi
q_en = translate.run({"text": raw_q})

docs = retriever.get_relevant_documents(q_en)
for d in docs[:5]:
    print("-", d.metadata.get("headings"), "| ts:",
d.metadata.get("ts"))
```

g. Normalization and spell-fix (cheap, local)

Fix common typos and normalize units so they match your docs.

Manual: write regex pre-processors before calling Pinecone.
LangChain: same code, but you tuck it in a tiny function before `retriever.get_relevant_documents()`—it composes cleanly with chains.

```
import re

def normalize_query(q: str):
    q = q.replace("celcius", "celsius").replace("miligrams",
"milligrams")
    q = re.sub(r"\bdeg\b", "degrees", q)
    q = re.sub(r"\b(\d+)\s*(deg|°c)\b", r"\1 °C", q, flags=re.I)
    return q

q = normalize_query("brew at 90 deg celcius for ashwagandha coffe")
docs = retriever.get_relevant_documents(q)
```

3. Retrieval of Candidates

Retrieval is where your pipeline decides *what evidence the model will see*. If query transformation is the steering wheel, retrieval is the **ingredient selection**: how many documents, from which namespace, with which filters, and in what order. We'll walk through the main retrieval modes you'll actually use—dense (vector), sparse (BM25), hybrid fusion, and re-ranking—always contrasting **manual** vs **LangChain**, and giving small, reusable code blocks.

3.1 Dense retrieval (vector DB → Pinecone)

What it is
Embed the query, do ANN search in Pinecone, get the top-k most similar chunks. Fast, great for semantic matches ("ashwagandha latte calming effects"), weaker on exact tokens (ratios, temperatures).

Manual
You embed, call `index.query`, pass `namespace` and optional `filter`, then sort by score.
LangChain
Wrap Pinecone with a vector store and ask for a retriever. It hides embedding + query plumbing.

```
vectorstore = PineconeVectorStore(index_name=INDEX_NAME,
embedding=embedding)
query = "What is Ashwagandha coffee?"
vectorstore.similarity_search(query)
```

3.2 Sparse Retrieval

What it is
Lexical matching that favors exact token overlap. Great for **numbers, units, ratios, times**, and when the user names a specific drink ("beaten coffee 1:2 ratio").

Manual (in-process BM25)
You create a local BM25 index over your chunks, then query it alongside Pinecone.

LangChain
Use `BM25Retriever` to build and query directly.

```
# ---- SPARSE-ONLY RETRIEVAL ----
retriever.alpha = 1.0  # 1.0 = sparse only (BM25), 0.0 = dense only
q = "Ashwagandha"
docs = retriever.get_relevant_documents(q)
print("Sparse-only hits:", len(docs))
for d in docs[:5]:
    print("-", d.metadata.get("source"), "|",
d.page_content[:120].replace("\n"," "))
```

Sparse Retrieval with Alpha Parameter

- The alpha parameter controls the balance between sparse (traditional term-based search like BM25) and dense (vector similarity) retrieval.
- Setting alpha = 1.0 configures the retriever to rely only on sparse retrieval, completely ignoring dense embeddings. Conversely, alpha = 0.0 switches retrieval to purely dense vector search.
- Values between 0 and 1 create a hybrid approach, combining sparse and dense signals, allowing users to fine-tune the retrieval method for their task.

How LangChain Simplifies Sparse Retrieval

- LangChain community's `PineconeHybridSearchRetriever` exposes this alpha weighting as a simple attribute on retrievers, so users can easily toggle or blend retrieval modes without rebuilding or replacing components.
- This flexibility enables seamless experimentation with sparse-only, dense-only, or hybrid retrieval in the same pipeline.
- Sparse retrieval benefits from mature, explainable term-matching techniques, useful when keywords or exact terms matter, while dense retrieval captures semantic similarity.

In summary, LangChain's alpha parameter provides an intuitive mechanism to switch or balance sparse and dense retrieval methods dynamically, empowering developers to optimize retrieval for their specific dataset and use case with minimal code changes.

3.3 Hybrid retrieval (dense + sparse fusion)

The alpha parameter in LangChain's retriever enables hybrid retrieval by balancing between sparse and dense search methods:

- When alpha is set between 0 and 1, the retriever performs a weighted combination of sparse retrieval (like BM25 keyword matching) and dense retrieval (semantic vector similarity).

- This means it leverages both exact keyword matches and semantic understanding simultaneously, improving recall and precision by capturing documents relevant either through keyword presence or conceptual similarity.
- For example, with alpha = 0.5, the retriever equally weighs sparse and dense signals, incorporating the strengths of both to return a richer and more diverse set of relevant documents.
- The parameter gives developers fine-grained control over retrieval behavior to optimize search performance for different datasets and query types without changing the underlying architecture.
- Combined with top_k, which limits the number of results, alpha-guided hybrid retrieval enables efficient, robust semantic search that adapts to user needs and content characteristics.

In essence, LangChain's alpha offers a simple yet powerful knob to tune between traditional keyword-based and modern semantic search, enabling hybrid retrieval workflows with agility and precision.

Manual

You wrote the normalizer and fusion function, then merged on `doc_id`.

LangChain

```
# --- QUERY: hybrid search (adjust alpha/k at runtime) ---
retriever.alpha = 0.5
retriever.top_k = 8
q = "turmeric coffee health benefits"
docs = retriever.get_relevant_documents(q)

print(f"Got {len(docs)} docs")
for d in docs[:5]:
    print("-", d.metadata.get("headings"), "| file:",
d.metadata.get("file_name"))
```

How to use & benefits

- Use when a query mixes **concept + exact** terms ("health benefits at 90 °C brew").
- Adjust `alpha` by domain; **0.5–0.7** dense is a good starting range.
- Keep sparse `k` broader than dense to let exact hits sneak in.

3.4 Re-ranking (precision boost on a short list)

What it is

After you assemble 20–40 candidates, run a cross-encoder re-ranker (or small LLM judge) to re-score each passage relative to the query, then keep the top 6–10. This significantly improves precision before you hand context to the LLM.

Manual

You pick a cross-encoder (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2`), score each `(q, passage)` pair, then sort.

Langchain

Reranking in LangChain is a powerful technique used to improve the quality of retrieved documents by applying a second stage of scoring, typically using a more precise but computationally expensive model like a Cross-Encoder.

How Reranking Works

- Initially, a retriever (dense, sparse, or hybrid) fetches a candidate set of documents based on similarity scores, often with a larger pool (e.g., top 20 or 25).
- A reranker, such as a Cross-Encoder model, then scores each candidate document more accurately by jointly encoding the query and document text. This approach captures deeper semantic or contextual relevance than the initial retrieval.
- The documents are subsequently sorted based on these reranker scores, producing a refined, higher-quality ranking of the top results (e.g., top 8).
- This separation keeps the initial retrieval efficient, while reranking improves precision, balancing speed and accuracy effectively.

LangChain and Reranking

- LangChain easily supports integrating rerankers with retrievers in a pipeline, enabling developers to build workflows that combine fast retrieval and expensive reranking transparently.
- Reranking can be applied after any retrieval method (dense, sparse, hybrid), boosting the results' relevance by considering query-document interactions more comprehensively.
- It enhances user experience by providing more accurate and trustworthy results, crucial in complex search or question-answering applications.

```
# pip install -qU sentence-transformers
from sentence_transformers import CrossEncoder

def get_candidates(q: str, k: int = 20, mode: str = "hybrid"):
    if mode == "dense":
        retriever.alpha = 0.0
    else:  # "hybrid"
        retriever.alpha = 0.5
```

```
    retriever.top_k = k
    return retriever.get_relevant_documents(q)

ce = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-2-v2")

def rerank_cross(q: str, docs, top_n: int = 8):
    scores = ce.predict([(q, d.page_content) for d in docs])
    ranked = sorted(zip(docs, scores), key=lambda x: x[1],
reverse=True)[:top_n]
    return [d for d, _ in ranked]

def show(title, docs, n=5):
    print(f"\n{title} ({len(docs)}):")
    for i, d in enumerate(docs[:n], 1):
        src = d.metadata.get("source", "<no-source>")
        s = d.page_content[:120].replace("\n", " ")
        print(f"{i:02d}. {src} | {s}")

q = "Ashwagandha coffee benefits"
cand_dense  = get_candidates(q, k=25, mode="dense")
cand_hybrid = get_candidates(q, k=25, mode="hybrid")

# BEFORE rerank
show("DENSE (before)",  cand_dense)
show("HYBRID (before)", cand_hybrid)

# AFTER rerank
top_dense  = rerank_cross(q, cand_dense,  top_n=8)
top_hybrid = rerank_cross(q, cand_hybrid, top_n=8)

show("DENSE (after CE)",  top_dense)
show("HYBRID (after CE)", top_hybrid)
```

4. Context Preparation

Once candidates are retrieved, the system must decide **which ones to keep and how to present them**. This step ensures the final prompt isn't flooded with noise or redundant chunks, and that the LLM receives the most **relevant, diverse, and compact evidence**.

If **retrieval is about finding everything possibly relevant**, context preparation is about

choosing the right few and formatting them for efficiency. Think of it as a chef who's selected ingredients (retrieval), but now trims, washes, and portions them before handing them to the cook (LLM).

4.1 Score Normalization & Fusion

What it is
Retrieved lists from different retrievers (dense, sparse, hybrid) use different scoring scales. Normalization brings them to a comparable range (e.g., [0,1]) so they can be combined fairly.

Manual
Write min-max or z-score normalization for each retriever's score output, then fuse (e.g., weighted average).

LangChain
Handled automatically in `EnsembleRetriever` (as seen earlier). When you chain retrievers, LC normalizes scores internally before weighting.

Benefits

- Consistent scoring across methods
- Easy experimentation with weights
- Prevents bias toward retrievers with larger raw score range.
- Much higher precision (LLM sees fewer off-topic passages)
- Easily adjustable top-n cutoff

4.2 Cleaning and Shortening

LangChain helps in cleaning and shortening retrieved documents through components like the ContextualCompressionRetriever, which uses an LLM-driven approach to compress or summarize long or noisy document content before presenting it to downstream pipelines.

How LangChain Enables Cleaning and Shortening

- After initial retrieval (often returning many or lengthy documents), LangChain wraps the base retriever with a compressor that leverages large language models to extract the most relevant, concise information from each document chunk.
- This LLM-based extractor filters out irrelevant details, removes redundancies, and produces a compressed summary or focused snippet that retains the essential meaning related to the query.

- It effectively reduces the text size and complexity, making the data easier for LLMs to consume in subsequent generation or reasoning steps without losing key context or insights.
- This approach is dynamic and query-aware, meaning compression adapts to the specific question rather than a generic summary, improving relevance.

Benefits of LangChain's Compression

- Helps reduce token usage and costs when feeding documents to large language models.
- Enhances response quality by minimizing noise and focusing on salient points.
- Supports handling long documents that exceed model input limits by presenting cleaner, shortened content.

In essence, LangChain's integration of LLM-powered compression and retrieval creates a robust pipeline for delivering clean, concise, and query-focused document insights efficiently.

Manual

Regex clean, truncate to N sentences, or apply a summarizer.

LangChain

Wrap retriever with a **DocumentCompressor** like `LLMChainExtractor`, which extracts the most query-relevant snippet from each passage.

```
from langchain.retrievers.document_compressors import
LLMChainExtractor
from langchain.retrievers import ContextualCompressionRetriever

# Base retriever = your Pinecone dense retriever
dense_ret = vectorstore.as_retriever(search_kwargs={"k": 12})

# Wrap with an LLM-based compressor (Gemini, OpenAI, etc.)
extractor = LLMChainExtractor.from_llm(llm)
compression_ret = ContextualCompressionRetriever(
    base_retriever=dense_ret,
    base_compressor=extractor
)

# Example query
q = "Compare turmeric vs saffron coffee for taste"
docs = compression_ret.get_relevant_documents(q)

for d in docs[:5]:
    print("-", d.metadata.get("headings"), "| file:",
```

```
d.metadata.get("file_name"))
```

4.3 Deduplication & Diversity Control

In LangChain, deduplication and diversity control are important strategies that complement embedding-based retrieval to improve the quality and usefulness of search results when working with vector stores like Pinecone.

Deduplication with LangChain

- LangChain helps avoid showing duplicate or near-duplicate documents by enabling application-level deduplication during retrieval or post-processing.
- This typically involves filtering out documents with identical content, metadata keys, or hash signatures after retrieving candidates from Pinecone.
- Since most vector stores (including Pinecone) do not inherently deduplicate content embeddings, LangChain provides flexible hooks or wrapper functions to apply deduplication based on document IDs, texts, or metadata.
- Deduplication enhances user experience by ensuring search results are unique and reducing redundant information.

Diversity Control with LangChain

- Diversity control focuses on returning a varied set of results that cover different facets, topics, or perspectives rather than overly similar items.
- LangChain supports this by allowing retrieval strategies or post-retrieval processing that apply diversity-promoting algorithms like query expansion, maximal marginal relevance (MMR), or custom re-ranking.
- This ensures the retrieved top-k documents differ sufficiently in content while still being relevant to the query.
- By integrating diversity control with vector search, LangChain enables more comprehensive and balanced answers, which is especially useful in exploratory search or generative AI contexts.

How the Provided Code Benefits from These

- Using embeddings from a robust model like GoogleGemini and storing them in Pinecone, LangChain allows retrieval of semantically rich vectors.
- Careful combination of retriever outputs with deduplication and diversity control methods improves the relevance and uniqueness of the top 20 results fetched (search_kwargs={"k": 20}).
- This ensures that users get a diverse and non-redundant set of documents aligned with the query's semantic intent.

In summary, LangChain provides the tools and architectural flexibility to layer deduplication and diversity control on top of embedding-based retrieval for high-quality, relevant, and varied search results within vector databases like Pinecone.

Manual
Cluster by source filename, drop near-duplicates with minhash/similarity check, cap max per source.

LangChain
Write a **custom post-processor** function (easy with LC's retriever composition).

```
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_pinecone import PineconeVectorStore

embeddings = GoogleGenerativeAIEmbeddings(
    model="models/text-embedding-004",
    google_api_key=os.environ["GEMINI_API_KEY"]
)
vectorstore = PineconeVectorStore(
    index_name=INDEX_NAME,   # your 384-dim MiniLM index
    embedding=embeddings,
    text_key="text",
)

base_retriever = vectorstore.as_retriever(search_kwargs={"k": 20})
```

```
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_pinecone import PineconeVectorStore
from langchain_community.document_transformers import
EmbeddingsRedundantFilter
from langchain.retrievers.document_compressors import
DocumentCompressorPipeline
from langchain.retrievers import ContextualCompressionRetriever

# Embeddings (must match your index dim)
# embeddings =
HuggingFaceEmbeddings(model_name="sentence-transformers/paraphrase-Mi
niLM-L6-v2")
# or reuse your existing `embeddings` object

vectorstore = PineconeVectorStore(index_name=INDEX_NAME,
```

```
embedding=embeddings, text_key="text")
base_retriever = vectorstore.as_retriever(search_kwargs={"k": 20})

# Dedup transformer -> wrap as a compressor
dedup_transformer = EmbeddingsRedundantFilter(embeddings=embeddings,
similarity_threshold=0.92)
compressor =
DocumentCompressorPipeline(transformers=[dedup_transformer])

dedup_retriever = ContextualCompressionRetriever(
    base_retriever=base_retriever,
    base_compressor=compressor,
)

docs = dedup_retriever.get_relevant_documents("Ashwagandha coffee
benefits")

for d in docs[:5]:
    print("-", d.metadata.get("headings"), "| file:",
d.metadata.get("file_name"))
```

```
from langchain_pinecone import PineconeVectorStore

# attach vector store
vs = PineconeVectorStore(index_name=INDEX_NAME, embedding=embeddings,
text_key="text")

# plain similarity (baseline)
ret_sim = vs.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 8}           # return top-8 by similarity
)

# MMR (diversity control)
ret_mmr = vs.as_retriever(
    search_type="mmr",
    search_kwargs={
```

```
        "k": 8,                          # final results
        "fetch_k": 40,                   # pool size to choose from
        "lambda_mult": 0.5               # 0=more diversity, 1=more
relevance
    }
)

q = "Ashwagandha coffee benefits"
docs_sim = ret_sim.get_relevant_documents(q)
docs_mmr = ret_mmr.get_relevant_documents(q)

# quick peek
def show(name, docs):
    print(f"\n{name} ({len(docs)}):")
    for i, d in enumerate(docs[:5], 1):
        print(f"{i:02d}.", d.metadata.get("source", "<no-source>"))

show("SIM", docs_sim)
show("MMR", docs_mmr)
```

4.4 Context Length Budgeting

LangChain helps with context length budgeting—managing the token limits of large language models—by providing tools and workflows to dynamically compress, filter, and select document content so it fits within strict token budgets without losing critical information.

How LangChain Supports Context Length Budgeting

- Candidate Retrieval with Generous k
  LangChain fetches a large pool of candidate documents or chunks (e.g., top 40) from vector stores like Pinecone, ensuring coverage but potentially exceeding token limits.
- **Document Compression Pipelines**
  Using components like DocumentCompressorPipeline, LangChain applies multiple transformers to clean, deduplicate, reorder, and filter documents:
    - EmbeddingsRedundantFilter removes near-duplicate chunks based on vector similarity.
    - EmbeddingsFilter (if available) selects the most relevant chunks to further trim the set.
    - LongContextReorder restructures content order for better coherence in summaries or generation.

- **Contextual Compression Retriever**
  This wrapper combines retrieval and compression, outputting shortened, focused document versions tailored to the query, reducing size while preserving meaning.
- **Token Counting and Hard Caps**
  LangChain enables integration of token counters (using tools like tiktoken) to measure and impose hard token limits on the final combined content, preventing input overflow in LLMs.
- **Selective Document Inclusion**
  Based on token estimates, LangChain filters documents to ensure the accumulated text stays within the specified token budget (e.g., 1200 tokens), guaranteeing compliance with model input constraints.

Benefits

- Enables handling of large document corpora or long contexts by dynamically compressing and selecting only essential content.
- Improves generation quality by ensuring LLM inputs are both relevant and within safe size limits.
- Automates complex document management and token budgeting workflows that would otherwise require manual tuning.

In summary, LangChain provides an integrated, modular solution for context length budgeting by combining intelligent compression, filtering, token estimation, and budgeting, allowing developers to build effective and scalable retrieval-augmented generation (RAG) pipelines that respect LLM token limits.

Manual

Estimate tokens per passage, drop or summarize until under budget.

LangChain

Use `TokenTextSplitter` or `RecursiveCharacterTextSplitter` to cut long passages; combine with compressor for auto-fit.

```
# FINAL: LangChain-only compression + token budget (with safe
fallbacks)

# Assumes you already have:
#   - INDEX_NAME (Pinecone index)
#   - embeddings  (HF MiniLM 384 or Gemini 768; matches your index)
# If not, define them before this block.

from langchain_pinecone import PineconeVectorStore
from langchain.retrievers import ContextualCompressionRetriever
```

```
from langchain.retrievers.document_compressors import
DocumentCompressorPipeline

# Built-in transformers (community pkg)
from langchain_community.document_transformers import (
    EmbeddingsRedundantFilter,   # drops near-duplicate chunks
    LongContextReorder,          # better ordering for long contexts
)

# Try to import EmbeddingsFilter (keep only the most relevant
chunks).
# If your LangChain version doesn't have it, we just skip it.
try:
    from langchain.retrievers.document_compressors import
EmbeddingsFilter
    HAS_EFILTER = True
except Exception:
    EmbeddingsFilter = None
    HAS_EFILTER = False

# 1) Base retriever (get a decent candidate pool)
vectorstore = PineconeVectorStore(index_name=INDEX_NAME,
embedding=embeddings, text_key="text")
base_retriever = vectorstore.as_retriever(search_kwargs={"k": 40})

# 2) Build compressor pipeline (dedup -> (optional) relevance filter
-> reorder)
transformers = [
    EmbeddingsRedundantFilter(embeddings=embeddings,
similarity_threshold=0.92),
]
if HAS_EFILTER:
    transformers.append(EmbeddingsFilter(embeddings=embeddings, k=30,
similarity_threshold=0.15))
transformers.append(LongContextReorder())

compressor = DocumentCompressorPipeline(transformers=transformers)

ccr = ContextualCompressionRetriever(
```

```
    base_retriever=base_retriever,
    base_compressor=compressor,
)

# 3) Retrieve compressed docs
query = "Ashwagandha coffee benefits"
compressed_docs = ccr.get_relevant_documents(query)

# 4) Hard token cap (uses tiktoken if available; else a rough
fallback)
try:
    import tiktoken
    enc = tiktoken.get_encoding("cl100k_base")
    def count_tokens(text: str) -> int:
        return len(enc.encode(text or ""))
except Exception:
    def count_tokens(text: str) -> int:
        # rough fallback (~4 chars per token)
        return max(1, len(text or "") // 4)

def cap_to_tokens(docs, max_tokens: int = 1200):
    total, kept = 0, []
    for d in docs:
        n = count_tokens(d.page_content)
        if total + n > max_tokens:
            break
        kept.append(d); total += n
    return kept, total

final_docs, used_tokens = cap_to_tokens(compressed_docs,
max_tokens=1200)

print(f"Compressed: {len(compressed_docs)}  |  Kept under budget:
{len(final_docs)}  |  tokens={used_tokens}")
for i, d in enumerate(final_docs[:5], 1):
    src = d.metadata.get("source", "<no-source>")
    snippet = (d.page_content or "").replace("\n", " ")[:120]  #
precompute to avoid backslash in f-string
    print(f"{i:02d}. {src} | {snippet}")
```

A **typical context prep pipeline**:

a) Retrieve ~40 candidates (dense + sparse)
b) Normalize scores, fuse lists
c) Re-rank with cross-encoder → top 8
d) Deduplicate & cap per source
e) Clean & shorten passages
f) Fit within context budget

Coffee Example:
Query: *"Health benefits of Ashwagandha latte"*
Without context prep → 10 almost identical "ashwagandha = adaptogen" chunks.
With prep → 2 unique chunks from different sources (science blog + recipe page), cleaned and summarized, ready for the prompt.

5. Prompt Construction

LangChain facilitates the creation of flexible, composable prompts that act as a bridge between retrieved, compressed document contexts and the large language model's (LLM) conversational generation—effectively connecting knowledge retrieval with natural language interaction.

How LangChain Enables Prompt Bridging

- LangChain's PromptTemplate and ChatPromptTemplate abstractions allow developers to build rich prompts with dynamic sections such as conversation history, compressed context blocks, and user questions—all integrated seamlessly in one input to the LLM.
- This templating system supports structured prompt composition, where retrieved documents are formatted into concise, citable context snippets that the LLM can directly reference.
- Developers can insert placeholders for prior dialogue messages, enabling multi-turn conversational context alongside external knowledge retrieval.
- LangChain also provides output parsers like StrOutputParser to cleanly extract the LLM's response and enforce guidelines (e.g., answering only from provided context and citing sources).
- Through its chaining operators (|), LangChain creates compact runnable pipelines that combine prompt creation, LLM call, and output parsing into a single expressive workflow.

- This bridge mechanism ensures that the information retrieved, compressed, and budgeted upstream is effectively communicated to the LLM, allowing it to generate accurate, context-aware answers grounded in retrieved data.

Benefits in Context Length Budgeting

- After reducing and filtering large documents to fit token budgets, LangChain's prompt system injects this optimized context directly into the LLM input, maximizing the value of limited context windows.
- Enables design of enterprise-grade retrieval-augmented generation (RAG) assistants that remain factually grounded, concise, and traceable thanks to explicit source citations.
- Balances information density, user history, and question clarity in the prompt, enhancing response quality and usefulness even with constrained token limits.

This is where you **shape what the LLM sees and how it must respond**. Good retrieval dies without a disciplined prompt. We'll go deep in code with two paths:

- **Manual**: you fully control message assembly, JSON schemas, citations, and budgets using `google.generativeai`.
- **LangChain**: compose prompts, memory, and structured outputs with `ChatPromptTemplate`, parsers, and runnables.

5.1 Making a normal prompt with Langchain

```
# pip install -qU langchain langchain-core langchain-google-genai

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser
from langchain.schema import HumanMessage, AIMessage

# 0) LLM (Gemini chat)
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.2,
)
```

```
# 1) Format retrieved docs into a compact, citable context block
def format_context(docs):
    lines = []
    for i, d in enumerate(docs, 1):
        src = d.metadata.get("source", d.metadata.get("file_name",
f"S{i}"))
        snippet = (d.page_content or "").strip().replace("\n", " ")
        lines.append(f"[S{i}] {src}\n{snippet}")
    return "\n\n".join(lines)

# 2) Sample history (replace with your tracked chat turns)
history = [
    HumanMessage(content="I like spiced lattes but want healthier
options."),
    AIMessage(content="We can explore adaptogenic coffee variants."),
]

# 3) Prompt with history + context
prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are Askly, an enterprise helpdesk assistant. "
     "Answer ONLY using the provided CONTEXT. If the answer is not in
CONTEXT, say so. "
     "Cite sources like [S1], [S2]. Be concise and actionable."
    ),
    MessagesPlaceholder(variable_name="history"),
    ("system", "CONTEXT:\n{context}"),
    ("human", "{question}")
])

# 4) Build the runnable chain
chain = prompt | llm | StrOutputParser()

# 5) Retrieve (or use your prepared context list)
# Example: ctx_docs = retriever.get_relevant_documents(query)
# If you already did budgetization/dedup, just pass that list
instead.
query = "What is Ashwagandha coffee and how does it taste?"
```

```
ctx_docs = retriever.get_relevant_documents(query)   # <- your
existing retriever
ctx_str  = format_context(ctx_docs)

# 6) Invoke
answer = chain.invoke({
    "history": history,
    "context": ctx_str,
    "question": query,
})

print(answer)
```

5.2 Practical notes

- Keep `context_block` **tight** (we already re-rank and cap per source). If you hit token limits, drop `max_total_chars` to ~5–6k in `build_context_block`.
- When you want **structured outputs** (JSON), add one line to the YAML (e.g., "Return JSON with fields …") and pair with a JSON parser in LC.
- If your model sometimes forgets to cite, add a **post-check** and a self-repair pass ("Insert [DocID] after each factual sentence you took from CONTEXT.").

6. Generation & Answering (LangChain-first)

This stage turns your **prompted context** into a **high-quality answer**—consistently, safely, and with structure you can trust. Below are the most useful LangChain patterns you'll actually use on top of your setup from Steps 1–4 and your YAML prompt (Step 5).

6.1 Core "prompt → model → parse" chain (JSON or text)

LangChain enables sophisticated generation capabilities by providing modular, composable chains that connect prompts, language models (LLMs), and output parsers, making it easy to control both the input and format of the generated response.

How LangChain Enables Generation

- Flexible Prompt Composition: LangChain offers templating tools like ChatPromptTemplate to build prompts that incorporate system instructions, conversation history, user questions, and formatting guidelines—allowing well-structured and context-aware inputs to the LLM.

- Model Integration: Language model wrappers (e.g., ChatGoogleGenerativeAI) enable seamless connection to various LLM APIs, abstracting calling details and providing consistent interfaces to send prompts and receive responses.
- Chaining Pipelines: Using a simple pipe (|) syntax, LangChain composes prompt templates, LLM calls, and output parsers into runnable chains, enabling end-to-end text generation workflows in concise, readable code.
- Output Parsing and Validation: LangChain supports parsers like StrOutputParser for plain text output or JsonOutputParser for structured JSON responses, which ensure the generated outputs conform to expected formats, improving reliability for applications that require strict data structures.
- Contextual and Stateful Interaction: With support for message placeholders and history tracking, LangChain supports multi-turn conversations, helping generate responses informed by prior user-LLM interactions.

Benefits

- Developers can create customized, goal-directed generation tasks, such as succinct answers or factual JSON data, by controlling prompt content and output shape.
- It supports validation and error handling of the generated content through output parsers, ensuring higher quality and structured outputs.
- LangChain's abstraction facilitates rapid experimentation with different prompt designs and models, accelerating development of reliable and consistent generative AI applications.

```
# pip install -qU langchain langchain-core langchain-google-genai

import os
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser,
JsonOutputParser

# --- LLM (Gemini chat) ---
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.2,
)
```

```
# (optional) a tiny fake history to show message placeholders
history = [
    ("human", "I like spiced lattes."),
    ("ai",    "Got it. We can look at turmeric or ashwagandha
coffee."),
]

# ------------------------------------------------------
# 1) TEXT PIPELINE: prompt → model → StrOutputParser
# ------------------------------------------------------
text_prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer succinctly and helpfully."),
    MessagesPlaceholder("history"),
    ("human", "{question}"),
])
text_chain = text_prompt | llm | StrOutputParser()

text_answer = text_chain.invoke({
    "history": history,
    "question": "What is turmeric coffee?",
})
print("TEXT:", text_answer)

# ------------------------------------------------------
# 2) JSON PIPELINE: prompt → model → JsonOutputParser
# ------------------------------------------------------
# The JsonOutputParser ensures the final output is valid JSON.
json_parser = JsonOutputParser()
format_instructions = json_parser.get_format_instructions()
# Tell the model exactly what keys to return.
json_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a factual assistant. "
     "Return ONLY a JSON object with keys: "
     "`answer` (string), `citations` (array of strings), `confidence`
(number 0..1)."),
    MessagesPlaceholder("history"),
    ("system", "FORMAT:\n{format_instructions}"),
    ("human", "{question}")
```

```
])

json_chain = json_prompt | llm | json_parser

json_answer = json_chain.invoke({
    "history": history,
    "format_instructions": format_instructions,
    "question": "List two benefits of turmeric coffee and cite
sources if mentioned in context.",
})
print("JSON:", json_answer)
```

6.2 Streaming to your UI (sync)

LangChain enables streaming to user interfaces (UI) by supporting streaming output from large language models (LLMs) and providing mechanisms to process and relay incremental token generation in real time.

How LangChain Supports Streaming to UI

- Streaming-enabled LLM Wrappers: LangChain integrates with LLMs that support token-by-token streaming (e.g., OpenAI, Google Gemini), allowing partial outputs to be received immediately as the model generates them rather than waiting for the full response.
- Callback Handlers: The framework provides callback interfaces where developers can plug in custom handlers to receive streaming tokens or events. These handlers can push tokens incrementally to UIs, websockets, or other real-time client endpoints.
- Low-latency Interaction: By sending tokens as they are generated, LangChain supports fluid conversational experiences, live text completion, and interactive applications that feel responsive and dynamic.
- Composable Streams: Streaming can be integrated into prompt chains or agent workflows, enabling seamless streaming of not only raw LLM completions but also structured outputs or multi-step generation processes.
- Framework and UI Integration: Through these streaming hooks, LangChain can easily connect backend LLM output to modern frontend frameworks (React, Vue, or Swift UI) or streaming APIs, facilitating real-time updates in chatbots, assistants, or dashboard apps.

Benefits

- Enhances user experience with real-time feedback and reduced wait times.
- Enables progressive result delivery for lengthy responses or complex generation tasks.

- Simplifies building interactive AI applications with seamless token flow from model to UI.

```
# assumes: text_prompt, llm, history already defined (from your last
code)
query = "What is turmeric coffee?"  # or reuse your `question`

text_runnable = text_prompt | llm   # drop the StrOutputParser while
streaming

buf = []
for chunk in text_runnable.stream({"history": history, "question":
query}):
    piece = getattr(chunk, "content", str(chunk))
    print(piece, end="", flush=True)
    buf.append(piece)
print("\n--- done ---")

full_text = "".join(buf)  # optional: the complete response
```

```
# assumes: json_prompt, json_parser, llm, history already defined
format_instructions = json_parser.get_format_instructions()
query = "List two benefits of turmeric coffee and cite sources."

json_runnable = json_prompt | llm   # stream raw JSON text; parse
AFTER
raw_buf = []
for chunk in json_runnable.stream({
    "history": history,
    "format_instructions": format_instructions,
    "question": query,
}):
    piece = getattr(chunk, "content", str(chunk))
    print(piece, end="", flush=True)
    raw_buf.append(piece)
```

```
raw_json = "".join(raw_buf)
print("\n--- parsing ---")
data = json_parser.parse(raw_json)   # dict
print(data)
```

6.3 Enforcing citations, style, and length (post-process)

LangChain helps with citations by providing structured mechanisms to enforce and validate source attribution within generated responses, ensuring answers are grounded and verifiable.

How LangChain Supports Citations

- Custom Output Validation: LangChain allows post-processing and validation of generated text using functions (like has_valid_cites) that check for proper citation formats referencing allowed source IDs, ensuring outputs cite only valid, known context pieces.
- Citation Repair Prompts: When citations are missing or incorrect, LangChain enables automatic generation of repair prompts that instruct the LLM to rewrite answers by explicitly adding valid citations without altering factual content, thus improving trustworthiness.
- Prompt Templates for Citation Guidelines: Developers can embed clear system-level instructions within prompts, guiding LLMs to consistently format citations (e.g., [S1], [S2]) linked to specific context snippets or document sources.
- Integration with Context Formatting: Using metadata-tagged context blocks (like [S1] source_name ...), LangChain connects source content and citations, allowing users to trace model answers back to original documents transparently.
- Output Parsing for Structured Attribution: Parsers like StrOutputParser facilitate clean extraction of citation-inclusive responses, supporting further application-level processing or UI display.

Benefits

- Enforces factual grounding by tying generated content to explicit, verified sources.
- Helps build transparent and reliable AI systems, critical for enterprise, legal, or research contexts.
- Simplifies implementing citation-aware conversational agents or summarizers that reference supporting materials clearly.

```
import re
from langchain_core.prompts import ChatPromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser

# assume you already have: llm, history, ctx_docs, text_prompt
allowed_ids = [f"S{i}" for i in range(1, len(ctx_docs)+1)]

def has_valid_cites(txt: str) -> bool:
    ids = re.findall(r"\[S(\d+)\]", txt or "")
    return bool(ids) and all(1 <= int(i) <= len(ctx_docs) for i in
ids)

# 1) get the initial text answer (from your text chain)
text_runnable = text_prompt | llm | StrOutputParser()
text_answer = text_runnable.invoke({"history": history, "question":
query})

# 2) repair if citations are missing/wrong
if not has_valid_cites(text_answer):
    repair_prompt = ChatPromptTemplate.from_messages([
        ("system",
         "You MUST add citations in the form [S#] that refer ONLY to
these allowed IDs: {allowed_ids}. "
         "Do not invent sources. If context lacks support, say: 'Not
found in context.'"),
        ("human", "Rewrite this answer to include the citations
without changing facts:\n\n{answer}")
    ])
    text_answer = (repair_prompt | llm | StrOutputParser()).invoke({
        "allowed_ids": ", ".join(allowed_ids),
        "answer": text_answer
    })

print(text_answer)
```

6.4 Answer self-check & repair (lightweight verifier)

How LangChain Implements Self-Check and Repair

1. Drafting an Initial Answer
   Using a prompt that instructs the LLM to answer strictly from provided context and cite sources, LangChain produces a first-pass draft answer, grounding output in supplied documents.
2. Automated Verification
   LangChain employs a separate verification prompt and structured JSON parser to compare the draft answer against the context. This step checks if all claims are supported, identifies problematic or uncited assertions, and produces a verdict including flags for repair needs.
3. Conditionally Triggered Repair
   If the verifier detects unsupported content or issues, LangChain invokes a repair prompt that requests the LLM to rewrite the answer. The LLM is instructed to remove or hedge unsupported parts, clearly state missing information, and ensure all claims are cited properly, producing a corrected final answer.
4. Controlled Repair Iterations
   The system can perform one or more repair cycles, balancing thoroughness with efficiency, and returns both the final refined answer and diagnostic information (draft, verdict, number of repairs).

Benefits of LangChain's Self-Check Workflow

- Improves factual accuracy and trustworthiness by systematically eliminating hallucinations or unsupported assertions.
- Enforces source grounding with explicit citation checks, promoting transparency.
- Modular design allows fine-grained control over each phase (draft, verify, repair), easily customizable with different LLM models or prompts.
- Automates quality assurance for generated answers without manual review, scalable for enterprise applications.
- Enhances user confidence by providing answers vetted against trusted documents and clearly flagging limitations.

```
# --- Lightweight self-check & repair (TEXT ONLY) ---

from langchain_google_genai  import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser,
JsonOutputParser
from langchain.schema import HumanMessage, AIMessage
```

```
# 0) LLM
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.2,
)

# 1) Helpers
def format_context(docs):
    lines = []
    for i, d in enumerate(docs, 1):
        src = d.metadata.get("source", d.metadata.get("file_name",
f"S{i}"))
        snippet = (d.page_content or "").strip().replace("\n", " ")
        lines.append(f"[S{i}] {src}\n{snippet}")
    return "\n\n".join(lines)

# 2) Draft prompt (answer from context only, with cites)
draft_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Answer ONLY using the provided CONTEXT. "
     "If the answer is not in CONTEXT, say so. "
     "Cite sources like [S1], [S2]. Be concise."),
    MessagesPlaceholder("history"),
    ("system", "CONTEXT:\n{context}"),
    ("human", "{question}")
])
draft_chain = draft_prompt | llm | StrOutputParser()

# 3) Verifier (JSON verdict)
verifier_parser = JsonOutputParser()
verifier_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a strict verifier. Compare the DRAFT_ANSWER to CONTEXT.
"
     "Return ONLY JSON with keys: "
     "`supported` (boolean), "
     "`problems` (array of strings describing unsupported/uncited
claims), "
```

```
     "`needs_repair` (boolean)."),
    ("system", "CONTEXT:\n{context}"),
    ("human",
"QUESTION:\n{question}\n\nDRAFT_ANSWER:\n{answer}\n\n{format_instruct
ions}")
])
verifier_chain = verifier_prompt | llm | verifier_parser

# 4) Repair prompt (produce final text answer)
repair_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Repair the answer so that EVERY claim is directly supported by
CONTEXT. "
     "Remove or hedge unsupported parts. If info is missing, say so
clearly. "
     "Cite sources like [S1], [S2]. Be concise."),
    ("system", "CONTEXT:\n{context}"),
    ("human",
     "QUESTION:\n{question}\n\nDRAFT_ANSWER (may be
flawed):\n{answer}\n\n"
     "Verifier flagged:\n{problems}\n\n"
     "Rewrite a corrected final answer.")
])
repair_chain = repair_prompt | llm | StrOutputParser()

# 5) One-call helper
def answer_with_self_check(question, ctx_docs, history=None,
max_repairs: int = 1):
    history = history or []
    ctx_str = format_context(ctx_docs)

    # draft
    draft = draft_chain.invoke({"history": history, "context":
ctx_str, "question": question})

    # verify
    verdict = verifier_chain.invoke({
        "context": ctx_str,
        "question": question,
```

```
        "answer": draft,
        "format_instructions":
verifier_parser.get_format_instructions(),
    })

    # repair if needed (once by default)
    repairs = 0
    final = draft
    if (not verdict.get("supported")) or verdict.get("needs_repair")
or verdict.get("problems"):
        if repairs < max_repairs:
            probs = "\n".join(verdict.get("problems") or [])
            final = repair_chain.invoke({
                "context": ctx_str,
                "question": question,
                "answer": draft,
                "problems": probs or "No details provided.",
            })
            repairs += 1

    return final, {"draft": draft, "verdict": verdict, "repairs":
repairs}

# ---- Example usage ----
history = [HumanMessage(content="I like spiced lattes."),
AIMessage(content="Let's explore turmeric." )]
ctx_docs = retriever.get_relevant_documents("Ashwagandha coffee
benefits")
final_text, debug = answer_with_self_check("Ashwagandha coffee
benefits", ctx_docs, history)
print(final_text)
```

Implementing RAGs

1. Memory RAG

1.1 What is “Memory-RAG” (in LangChain terms)?

Plain RAG grounds answers in documents but forgets the user. **Memory-RAG** adds a second evidence stream—user facts, preferences, and recent session context—so the model reasons over both: long-term docs and short-term, user-specific memory.

1.2 Two evidence stores, two roles

- **Knowledge store (docs):** relatively static truth (our coffee pages). Optimized for semantic retrieval and exact terms (dense + optional sparse).
- **User memory store:** tiny, high-signal facts about the user/session (e.g., "prefers saffron, avoids dairy"). It's a separate namespace/index to keep retrieval cheap and precise.

1.3 History ≠ Memory

History is the raw transcript (messages). It grows fast and is not directly searchable.
Memory is distilled from history (salient facts only), written to a small vector store. We read **history** for conversation flow, and **memory** for factual personalization.

1.4 Retrieval contract

On each turn, we perform two queries:

- **Docs retriever:** top-k passages about the topic.
- **Memory retriever:** top-k user facts.
  We then **merge and format** them into distinct prompt sections: `CONTEXT:` (docs) and `RELEVANT_MEMORY:` (user facts). This keeps provenance clear and reduces leakage between sources.

1.5 Code

a. Setup (keys, models, vector stores, helpers)

```
# !pip install -q langchain langchain-community
langchain-google-genai sentence-transformers

import os
from pathlib import Path
from textwrap import shorten

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_community.embeddings import HuggingFaceEmbeddings
```

```
from langchain_community.vectorstores import Pinecone as LC_Pinecone
from langchain_core.documents import Document

# ---- Config (edit if needed) ----
INDEX_NAME = "coffeeindex"
DOCS_NS    = "docs"
USER_ID    = "user_001"

# Keys (no ADC)
os.environ["PINECONE_API_KEY"] =
Path("api_key.txt").read_text(encoding="utf-8").strip()
GEMINI_KEY =
Path("gemini_key.txt").read_text(encoding="utf-8").strip()

# LLM (Gemini)
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=GEMINI_KEY,
    transport="rest",
    temperature=0,
)

# Vector store (assumes embeddings already upserted earlier)
embedding   =
HuggingFaceEmbeddings(model_name="sentence-transformers/paraphrase-Mi
niLM-L6-v2")
vstore_docs = LC_Pinecone.from_existing_index(index_name=INDEX_NAME,
embedding=embedding, namespace=DOCS_NS)

# Helper: build CONTEXT text with [DocID]
def context_block(docs, max_chars=400):
    if not docs:
        return "(no context)"
    lines = []
    for d in docs:
        md = d.metadata or {}
        did = md.get("doc_id") or
f"{md.get('file_name','src')}#{md.get('chunk_id','?')}"
        clean = (d.page_content or "").replace("\n", " ")
```

```
        lines.append(f"[{did}] {shorten(clean, max_chars,
placeholder=' ...')}")
    return "\n".join(lines)
```

b. Prompt with separate chat history

```
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import
RunnableWithMessageHistory
from langchain_community.chat_message_histories import
ChatMessageHistory

# Chat history storage (per session_id)
_histories = {}
def get_session_history(session_id: str) -> ChatMessageHistory:
    if session_id not in _histories:
        _histories[session_id] = ChatMessageHistory()
    return _histories[session_id]

# Prompt uses HISTORY + QUESTION + CONTEXT (docs only)
prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a helpful coffee expert. Use ONLY CONTEXT to answer.\n"
     "If the answer is not in CONTEXT, reply exactly: I don't know
from the provided documents.\n"
     "Cite sources by their [DocID] when you use them. Keep answers
concise."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "QUESTION:\n{query}\n\nCONTEXT:\n{context_block}")
])

# Base chain
base_chain = prompt | llm

# Wrap with message history:
# - input_messages_key must match the user input variable ("query")
```

```
# - history_messages_key must match the MessagesPlaceholder
("history")
chain_with_history = RunnableWithMessageHistory(
    base_chain,
    get_session_history,
    input_messages_key="query",
    history_messages_key="history",
)
```

c. Retrieve document context (dense) and build inputs

```
# Dense retrieval from your docs index
def retrieve_docs(q: str, k: int = 8, filt: dict | None = None):
    retr = vstore_docs.as_retriever(search_kwargs={"k": k, "filter":
filt} if filt else {"k": k})
    return retr.get_relevant_documents(q)

# One call: retrieve -> format context -> run chain (history handled
automatically)
def ask(query: str, k: int = 8, session_id: str = USER_ID):
    docs = retrieve_docs(query, k=k)
    ctx = context_block(docs)
    resp = chain_with_history.invoke(
        {"query": query, "context_block": ctx},
        config={"configurable": {"session_id": session_id}},
    )
    answer = getattr(resp, "content", resp)
    print(f"\nAssistant:\n{answer}\n")
    return docs, answer

# Quick smoke test (non-interactive)
_ = ask("What is beaten coffee and its typical ratio?", k=6)
```

d. Interactive loop with persistent history

```
print("Type your question (or 'exit' to quit):")
while True:
    try:
        q = input("You: ").strip()
    except (EOFError, KeyboardInterrupt):
        break
    if not q or q.lower() in {"exit", "quit"}:
        break
    ask(q, k=8, session_id=USER_ID)
print("Bye!")
```

e. Inspect / reset chat history

```
hist = get_session_history(USER_ID).messages
print(f"History has {len(hist)} messages:")
for m in hist[-6:]:
    role = getattr(m, "type", getattr(m, "role", ""))
    print(f"- {role.upper()}: {getattr(m, 'content', m)[:120]}")

# To reset history without touching your document store:
# _histories[USER_ID] = ChatMessageHistory()
```

How this "Memory-RAG" works here

- **Docs retriever** (Pinecone via LangChain) supplies `CONTEXT` per user query.
- **Chat history** is **separate** and stored in-memory via `RunnableWithMessageHistory` → it injects previous turns into the `{history}` slot, not your vector DB.
- The LLM is instructed to use **only** `CONTEXT` for facts (so it cites docs) but can use **history** for conversational continuity (follow-ups, pronouns, etc.).
- No user memory is embedded; you can add a second retriever for profile facts later if needed, but this keeps your requirement: **history & model replies separate from docs** while still doing RAG.

2. Self Adaptive RAG

2.1 What "Self-Adaptive RAG" means (policy, not plumbing)

A self-adaptive RAG doesn't run a fixed pipeline every time. It **chooses retrieval knobs at runtime** from the query itself:

- **Planner** reads the query and infers scope:
  - Broad/comparative ("compare, guide, vs, how…") → **wider recall** (bigger `k0`), **rerank** + **compression**, bigger context budget.
  - Numeric/factual (ratios, temperatures, years) → **moderate recall**, **rerank**, **no compression** (to avoid clipping numbers).
  - Otherwise → balanced defaults.
- **Precision vs. Recall:** The planner turns on/off **cross-encoder re-ranking** and **LLM compression** to shape evidence before prompting.
- **Adaptive retry:** If the first pass looks weak (low confidence / "I don't know"), the system **widens recall and budget**, re-answers, and keeps the better result.

Net effect: the pipeline **spends tokens where they matter** and **stays cheap** for simple asks.

2.2 Evidence shaping (rerank → compress → budget)

After initial dense retrieval:

- **Re-rank (CrossEncoder)** re-scores each (query, passage) pair to upgrade precision on the shortlist.
- **Compression (LLM extractor)** keeps *only the spans* needed to answer the question (especially useful for broad asks).
- **Context budgeting** trims each kept passage and greedily packs into a fixed token budget. You still **preserve DocIDs** in a separate display block so citations remain intact even if content is trimmed.

2.3 Guardrails & adaptive retry (verification loop)

Before returning:

- **Verify head** checks grounding ("list factual errors or missing citations; return 'OK' if none").
- **Confidence head** rates confidence from 0–1 using *context only*.
- **Retry policy:** On low confidence or "I don't know from the provided documents." →

widen plan (larger `k0`, more `top_m`, no compression, larger budget) and **re-answer once**. Keep the higher-confidence result.

a. Setup (keys, LLM, vector store, helpers)

```
# !pip install -q langchain langchain-community
langchain-google-genai sentence-transformers pyyaml

import os, math, json, time
from pathlib import Path
from textwrap import shorten

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Pinecone as LC_Pinecone
from langchain_core.documents import Document

# ---- Config (adapt as needed) ----
INDEX_NAME = "coffeeindex"
DOCS_NS    = "docs"

# Keys (no ADC)
os.environ["PINECONE_API_KEY"] =
Path("api_key.txt").read_text(encoding="utf-8").strip()
GEMINI_KEY =
Path("gemini_key.txt").read_text(encoding="utf-8").strip()

# LLM (Gemini)
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=GEMINI_KEY,
    transport="rest",
    temperature=0,
)

# Vector store (assumes dense embeddings already upserted)
embedding   =
HuggingFaceEmbeddings(model_name="sentence-transformers/paraphrase-Mi
niLM-L6-v2")
```

```
vstore_docs = LC_Pinecone.from_existing_index(index_name=INDEX_NAME,
embedding=embedding, namespace=DOCS_NS)

# ---- Context formatting helper ----
def to_context_block(docs, max_chars=420):
    if not docs: return "(no context)"
    lines=[]
    for d in docs:
        md = d.metadata or {}
        did = md.get("doc_id") or
f"{md.get('file_name','src')}#{md.get('chunk_id','?')}"
        clean = (d.page_content or "").replace("\n"," ")
        lines.append(f"[{did}] {shorten(clean, max_chars,
placeholder=' ...')}")
    return "\n".join(lines)

# Tiny token estimator + trimming (naive char/4)
def est_tokens(txt: str) -> int: return max(1, len(txt)//4)
def trim_to_tokens(txt: str, max_tok: int) -> str: return
txt[:max_tok*4]
```

b. Planner (self-adaptive knobs) + retrieval

```
from dataclasses import dataclass

@dataclass
class Plan:
    k0: int                # initial top-k for dense retrieval
    top_m: int             # how many to keep after rerank/compress
    use_rerank: bool
    use_compress: bool
    budget_tokens: int     # total context budget (tokens)
    per_doc_cap: int       # cap per doc (tokens)

def plan_query(q: str) -> Plan:
    ql = q.lower()
    broad = any(w in ql for w in ["compare", "guide", "how", "steps",
```

```
"vs", "advantages", "disadvantages"])
    numeric = any(ch.isdigit() for ch in ql) or ":" in ql
    # Simple heuristics
    if broad:
        return Plan(k0=40, top_m=12, use_rerank=True,
use_compress=True, budget_tokens=2200, per_doc_cap=280)
    if numeric:
        return Plan(k0=24, top_m=8,  use_rerank=True,
use_compress=False, budget_tokens=1600, per_doc_cap=260)
    return Plan(k0=32, top_m=10, use_rerank=True, use_compress=True,
budget_tokens=1800, per_doc_cap=260)

# Dense retrieval
def dense_retrieve(query: str, k: int = 32, filt: dict | None =
None):
    retriever = vstore_docs.as_retriever(search_kwargs={"k": k,
"filter": filt} if filt else {"k": k})
    return retriever.get_relevant_documents(query)

# With scores (for budgeting/rerank seed)
def dense_with_scores(query: str, k: int = 32, filt: dict | None =
None):
    return vstore_docs.similarity_search_with_score(query, k=k,
filter=filt)
```

c. Rerank (CrossEncoder), compression (LLM), and context budgeting

```
# !pip install -q sentence-transformers
from sentence_transformers import CrossEncoder
from langchain.retrievers.document_compressors import
LLMChainExtractor

# Cross-encoder for reranking (fast, good)
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(query: str, docs, top_m: int = 10):
    pairs = [(query, d.page_content) for d in docs]
```

```
    scores = reranker.predict(pairs).tolist()
    order = sorted(range(len(docs)), key=lambda i: scores[i],
reverse=True)[:top_m]
    return [docs[i] for i in order], [scores[i] for i in order]

# Compression (keep only relevant spans)
extractor = LLMChainExtractor.from_llm(llm)
def compress_docs(query: str, docs):
    return extractor.compress_documents(docs, query=query)

# Budget packer: greedy by given order, trims long docs
def pack_docs(docs, max_ctx_tokens=1800, per_doc_cap=280,
sep="\n\n---\n\n"):
    budget = max_ctx_tokens
    out_parts, used = [], 0
    for d in docs:
        txt = d.page_content or ""
        txt = trim_to_tokens(txt, per_doc_cap)
        need = est_tokens(txt) + (est_tokens(sep) if out_parts else
0)
        if used + need > budget:
            # try partial fit
            remain = budget - used - (est_tokens(sep) if out_parts
else 0)
            if remain <= 0: break
            txt = trim_to_tokens(txt, remain)
            if not txt: break
        if out_parts: out_parts.append(sep)
        out_parts.append(txt)
        used += est_tokens(txt) + (est_tokens(sep) if
len(out_parts)>1 else 0)
    return "".join(out_parts), used
```

d. Answering with verification + adaptive retry

from langchain.prompts import ChatPromptTemplate, PromptTemplate
import re

```
# Base prompt (docs only; no hallucinations)
prompt_ans = ChatPromptTemplate.from_messages([
```

```
    ("system",
     "You are a helpful coffee expert. Use ONLY CONTEXT to answer.\n"
     "If not in CONTEXT, reply exactly: I don't know from the
provided documents.\n"
     "Cite sources by their [DocID]. Keep 2-6 sentences."),
    ("human", "QUESTION:\n{q}\n\nCONTEXT:\n{ctx}")
])

# Verify & confidence heads
verify_p = PromptTemplate.from_template(
    "Given CONTEXT and ANSWER, list factual errors or missing
citations. Return 'OK' if none.\n\nCONTEXT:\n{ctx}\n\nANSWER:\n{ans}"
)
conf_p = PromptTemplate.from_template(
    'Rate confidence 0.0-1.0 based ONLY on CONTEXT. Return JSON:
{"confidence": float, "rationale":
"short"}\n\nCONTEXT:\n{ctx}\n\nANSWER:\n{ans}'
)

def enforce(answer: str, max_sents=6):
    parts = re.split(r"(?<=[.!?])\s+", answer.strip())
    return " ".join(parts[:max_sents]) if parts else answer.strip()

def answer_once(q: str, plan: Plan, filt: dict | None = None):
    # 1) retrieve
    hits = dense_with_scores(q, k=plan.k0, filt=filt)
    docs = [d for d,_ in hits]
    if not docs: return "(no context)", "(no answer)", 0.0, "no docs"

    # 2) rerank
    if plan.use_rerank:
        docs, _ = rerank(q, docs, top_m=plan.top_m)
    else:
        docs = docs[:plan.top_m]

    # 3) compress
    if plan.use_compress:
        docs = compress_docs(q, docs)
```

```
    # 4) pack & render context
    ctx_text, _ = pack_docs(docs, max_ctx_tokens=plan.budget_tokens,
per_doc_cap=plan.per_doc_cap)
    ctx_block = to_context_block([Document(page_content=t,
metadata=d.metadata) for t,d in zip([d.page_content for d in docs],
docs)]) \
                if docs else "(no context)"
    # Use the packed text for model, but still show ids via ctx_block
if needed
    model_ctx = ctx_text if ctx_text.strip() else
to_context_block(docs)

    # 5) answer
    msg = (prompt_ans | llm).invoke({"q": q, "ctx": model_ctx})
    ans = enforce(getattr(msg, "content", msg), max_sents=6)

    # 6) verify & confidence
    issues = (verify_p | llm).invoke({"ctx": model_ctx, "ans":
ans}).content.strip()
    conf_msg = (conf_p | llm).invoke({"ctx": model_ctx, "ans": ans})
    try:
        conf = json.loads(getattr(conf_msg, "content",
conf_msg)).get("confidence", None)
    except Exception:
        conf = None
    return model_ctx, ans, conf if conf is not None else 0.0, issues

def adaptive_answer(q: str):
    plan = plan_query(q)
    ctx, ans, conf, issues = answer_once(q, plan)

    # Retry rule: if low confidence or model said "I don't know",
relax and retry once
    low_conf = conf is None or conf < 0.5
    unknown  = "I don't know from the provided documents." in ans
    if low_conf or unknown:
        widened = Plan(
            k0=plan.k0 + 24,
            top_m=min(plan.top_m + 4, plan.k0 + 24),
```

```
            use_rerank=True,
            use_compress=False,   # turn off compression to keep more
signal
            budget_tokens=plan.budget_tokens + 600,
            per_doc_cap=plan.per_doc_cap
        )
        ctx2, ans2, conf2, issues2 = answer_once(q, widened)
        # pick the better one
        if (conf2 or 0) >= (conf or 0):
            return widened, ctx2, ans2, conf2, issues2
    return plan, ctx, ans, conf, issues
```

e. Try it (single call or tiny REPL)

```
# Single question demo
Q = "Compare turmeric vs saffron coffee for taste and digestion, with
non-dairy ideas."
plan, ctx_used, answer, confidence, issues = adaptive_answer(Q)

print("PLAN:", plan)
print("\nANSWER:\n", answer)
print("\nCONFIDENCE:", confidence)
print("VERIFY:", issues)

# --- Optional: tiny interactive loop ---
# print("\nType your question (or 'exit'):")
# while True:
#     q = input("You: ").strip()
#     if not q or q.lower() in {"exit","quit"}: break
#     plan, ctx_used, answer, confidence, issues = adaptive_answer(q)
#     print("\nPlan:", plan)
#     print("\nAssistant:", answer)
#     print("Confidence:", confidence, "| Verify:", issues, "\n")
```

Conclusion

Indexing & Embeddings – We ingested HTML coffee documents, enriched them with metadata (titles, headings, keywords), chunked them, and stored dense embeddings in **Pinecone** for semantic search.

Retrieval with Metadata – We showed how to upsert metadata (e.g., `heading_keywords`, timestamps), and apply filters in queries for more targeted retrieval.

Dense & Sparse Retrieval – Implemented dense retrievers with Pinecone, BM25 sparse retrieval, and even hybrid setups, giving multiple retrieval strategies.

Reranking – Used **CrossEncoder models** to rerank dense candidates, improving result precision by aligning to the query semantics.

Document Post-processing – Added **deduplication, diversity control, context budgeting, and meta-sensitive prep** to ensure balanced, non-redundant, token-efficient context construction.

Prompt Construction – Built YAML-based reusable prompts, integrated with LangChain, and emphasized context injection, citation rules, and memory blocks for personalization.

Memory-RAG – Implemented conversational memory using LangChain's `RunnableWithMessageHistory`, keeping **chat history separate** from the docs while grounding answers in the retrieved context.

Self-Adaptive RAG – Added intelligence: a planner adjusts retrieval depth, reranking, compression, and context size per query, plus confidence scoring, verification, and retries when the model is uncertain.

Advanced Prompt Techniques – Demonstrated structured JSON/text parsing, streaming, style/length enforcement, verification loops, rationale capture, routing between small/big models, and retries.

Takeaway – RAG isn't just "retrieve + answer." A **production-grade RAG** needs layers: careful document prep, retrieval diversity, reranking, context management, adaptive pipelines, prompt engineering, and conversational memory. Together, these deliver answers that are **relevant, reliable, efficient, and grounded in sources**.